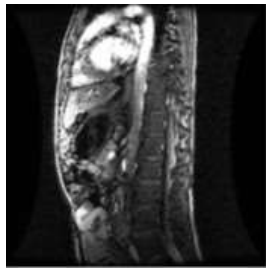


영상 향상(Image Enhancement)

1) 설명

화소 변환 및 히스토그램 처리 과정 등을 적용하여 시각적으로 향상된 결과 영상을 만든다. 아래와 그림과 같이 제시된 입력 영상에 목표를 정의하고 학습 시간에 배운 처리 방법을 적용하여 향상된 출력 영상을 얻는다.

2) 예시

입력 영상	목표 정의	출력 영상
	● 화소 레벨의 밝기를 조절하기 ● 선명도가 낮아 선명도를 높이기 ● 구별되는 영역을 상대적으로 강조하기 ● ...	

3) 요구사항

- 영상 향상의 정량적 또는 정성적 목표를 기술한다.
- 구현 방법은 파이썬에서 제공되는 다양한 라이브러리 함수를 활용한다.
- 학습 시간에 배운 처리 방법을 적용하되, 적용 기준 또는 이유를 설명한다.
- 처리 방법 적용 후, 결과 영상의 향상 정도를 정량적 또는 정성적으로 분석한다.
- 입력에 활용된 영상의 출처를 반드시 밝힌다.
- 영상 처리 방법을 직접 구현하지 않고, 외부의 소스 또는 프로그램을 활용한 경우, 출처를 밝히고, 자신이 새롭게 알게 된 절차, 추가 및 개선한 점을 서술한다.

4) 평가 기준

- 영상 입출력과 처리 방법 이해도 (20%)
- 향상 목표 정의의 타당성 및 독창성 (30%)
- 분석 및 과정 서술의 구체성 (30%)
- 최종 구현의 완성도 (20%)

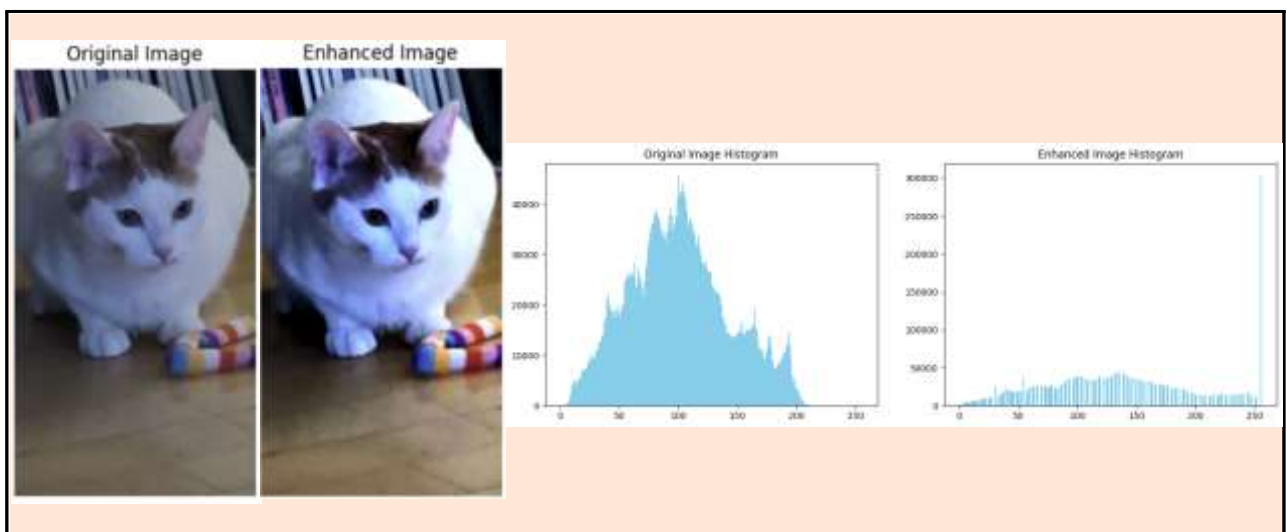
과제 보고서

주제명	직접 촬영한 사진 이미지의 화질 개선을 위한 방법 탐구	
작성자	성명	E-mail
	권영훈	20213032hun@gamil.com

< 입력과 목표 정의 >

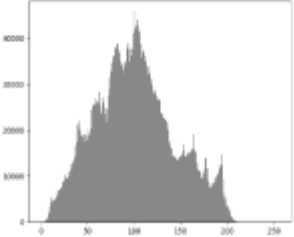
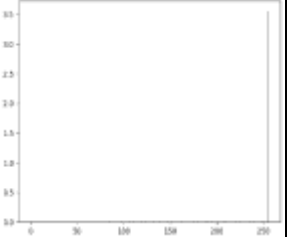
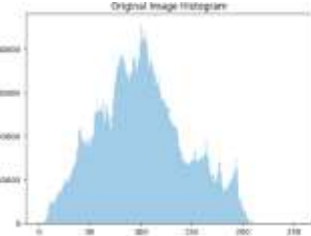
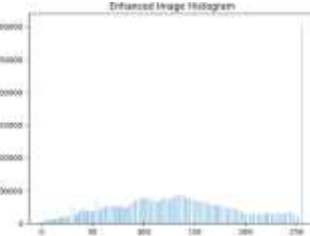
입력 영상	목표 정의
	1. 이미지 밝기 증가 2. 선형 대비 늘리기 3. 화소 변환 - 감마 보정 4. 결과 값 비교를 위해 히스토그램 사용.

< 대표 출력 결과 >



(지면 부족시 추가하여 작성)

〈 목표 정의 〉

설명	휴대폰 카메라로 찍은 사진의 화질이 좋지 않아 사진의 화질(선명도)을 향상하고 싶음.
접근 방법	1. 이미지의 밝기를 증가시키기 위해 사용 - (인터넷 검색: 밝기 조절을 위해 넘파이 함수 (np.clip) 사용) 2. 이미지의 선형 대비를 향상시키기 위해 사용 - (강의자료 참조: python 코드의 선형 대비 늘리기 함수 사용) 3. 사용자의 보기 편한 이미지 제공을 위해 감마 보정 사용 - (인터넷 검색: 사람의 눈에 더 자연스럽게 보이기 위해 감마 보정 사용) 4. 원본 이미지와 향상된 이미지 결과를 비교하기 위해 히스토그램 사용 - (강의자료 참조: python 코드의 히스토그램 코드 사용)
결과 및 분석	 ->    <pre style="background-color: #f0f0f0; padding: 10px; margin: 10px 0;">def ContrastStretch(px): # 선형 대비 늘리기 함수 r1, r2 = 10, 70 if px < r1: return 0 elif px > r2: return 255 else: return 255 * (px - r1) / (r2 - r1)</pre> 1. 교수님의 파이썬 코드 내 선형 대비 늘리기 함수의 r1, r2의 값에 초기 설정 값인 10과 70은 결과물이 너무 밝음. (결과 이미지가 극단적으로 밝고, 명암 대비가 너무 강해져서 거의 흰색으로 변함.) 2. 초기 설정 값의 결과 히스토그램 분포에도 픽셀이 특정 한 지점(255)의 높은 그래프 수치 하나를 제외한 값은 표시되지 않음. 3. 이 후 여러 번의 테스트 후 최적 값(6, 255)을 찾음. (아래는 최적 값의 결과 이미지와 히스토그램)  Original Image  Enhanced Image  

(지면 부족시 추가하여 작성)

(밝기 향상)

```
enhanced_img_array = np.clip(img_array * 1.5, 0, 255).astype(np.uint8)
```

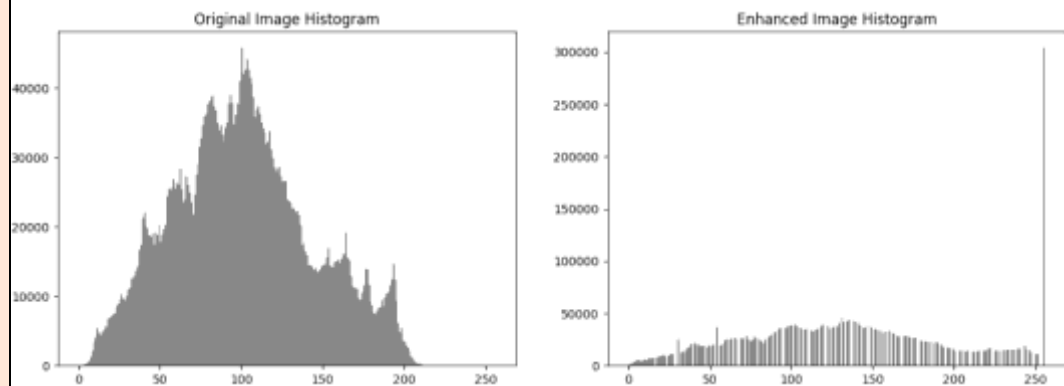
1. 원본 이미지의 모든 픽셀 값을 1.5배 증가시켜 밝기를 향상함.
2. 몇 번의 테스트 후 최적 값이 초기 1.5 값이라는 것을 찾음.

(화소 변환) - 감마 보정

```
gamma = 1.2
```

```
gamma_corrected = np.clip((contrast_stretched / 255) ** gamma * 255, 0, 255).astype(np.uint8)
```

- 감마 보정: 이미지의 밝기와 대비를 조정
 - 감마 값은 1보다 크면 어두운 영역을 더 밝게 만들고, 1보다 작으면 밝은 영역을 더 강조하여 전체적으로 어둡게 만듦.
 - gamma 값을 1.2로 설정함.
1. (contrast_stretched / 255): 이전 단계(0~255 범위의 화소 값)값을 255로 나누기.
 2. (contrast_stretched / 255) ** gamma * 255: 각 픽셀 값에 gamma 승 연산에 255 곱.
 3. np.clip(..., 0, 255): 변환된 픽셀 값이 0~255 범위를 초과하지 않도록 제한함.
 4. .astype(np.uint8): 이미지 데이터로 저장



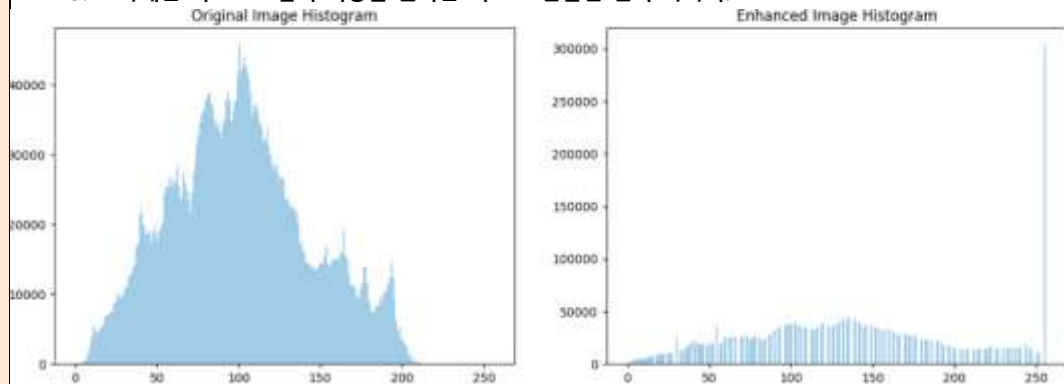
1. 히스토그램의 색상을 변경하고 싶음.

```
plt.hist(img_array.flatten(), bins=256, range=(0, 256), color='gray')
```

2. 색상 값을 'gray'에서 원하는 색인 'skyblue'로 변경함.

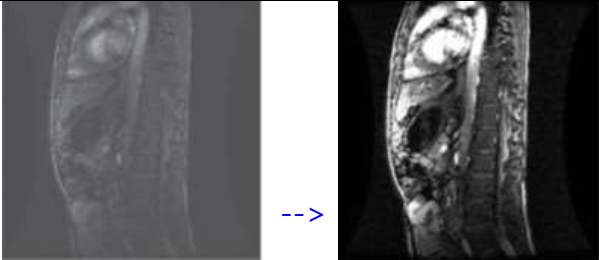
```
plt.hist(img_array.flatten(), bins=256, range=(0, 256), color='skyblue')
```

3. 아래는 히스토그램의 색상을 원하는 색으로 변환한 결과 이미지.



(지면 부족시 추가하여 작성)

< 목표 정의 예 >

설명	어두운 화면을 위한 밝기 향상
접근 방법	- 영상의 화소 밝기 값의 배수(곱하기)를 통한 밝기 변환 (강의자료 참조: python 코드의 PIL 라이브러리 ... 함수 사용) - 관심 제외 대상 밝기 값 제외 (인터넷 검색: skimage 라이브러리의 crop 사용)
결과 및 분석	 - 너무 큰 배수 곱에 의한 밝기 변환은 영상의 치우침 현상이 발견되어, 약 1.5배의 곱의 최적값을 찾음 - 영상 크기를 줄임으로 인해서 원본 이미지와의 비교가 어려울 수 있음

< 참고 문헌: URL, 도서 등 >

- 1) 넘파이 함수를 이용한 밝기 조절 - <https://deep-learning-study.tistory.com/114>
- 2) 감마 조절 - <https://marisara.tistory.com/entry/%ED%8C%8C%EC%9D%B4%EC%8D%AC-openCV-2-%EA%B0%90%EB%A7%88-%EB%B3%B4%EC%A0%95Gamma-Correction>

< 사진 출처 및 소스코드 >

■ 사진 출처: 본인

■ 고양이: 반려 묘 치즈

■ 소스코드:



```
from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

def ContrastStretch(px): # 선형 대비 늘리기 함수
    r1, r2 = 10, 255
    if px < r1:
        return 0
```

```
elif px > r2:
    return 255
else:
    return 255 * (px - r1) / (r2 - r1)
```

```
image_path = '/content/고양이.jpg'
```

```
try:
```

```
    # 이미지 열기
```

```
    img = Image.open(image_path)
    img_array = np.array(img)
    # 원본 이미지 표시
```

```
    plt.figure(figsize=(10, 5))
    plt.subplot(121)
    plt.imshow(img)
    plt.title("Original Image")
    plt.axis('off')
```

```
    # 밝기 향상
```

```
    enhanced_img_array = np.clip(img_array * 1.5, 0, 255).astype(np.uint8)
```

```
    # 선형 대비 늘리기
```

```
    vectorized_stretch = np.vectorize(ContrastStretch)
    contrast_stretched = vectorized_stretch(enhanced_img_array).astype(np.uint8)
```

```
    # 화소 변환 - 감마 보정
```

```
    gamma = 1.2
    gamma_corrected = np.clip((contrast_stretched / 255) ** gamma * 255, 0,
255).astype(np.uint8)
```

```
    # 결과 이미지 표시
```

```
    enhanced_img = Image.fromarray(gamma_corrected)
    plt.subplot(122)
    plt.imshow(enhanced_img)
    plt.title("Enhanced Image")
    plt.axis('off')
    plt.show()
```

```
    # 히스토그램 표시
```

```
    plt.figure(figsize=(15, 5))
    plt.subplot(121)
    plt.hist(img_array.flatten(), bins=256, range=(0, 256), color='skyblue')
    plt.title("Original Image Histogram")
```

```
    plt.subplot(122)
    plt.hist(gamma_corrected.flatten(), bins=256, range=(0, 256), color='skyblue')
    plt.title("Enhanced Image Histogram")
    plt.show()
```

```
except FileNotFoundError:
```

```
    print(f"Error: 이미지 파일 '{image_path}'을 찾을 수 없습니다.")
```